

Homework 1 解答

线性代数、概率、最小二乘与 NumPy

Problem 1: 线性代数复习

(a) 证明转置恒等式

设 A 的第 i 行为 a_i^T , 其中 $a_i \in \mathbb{R}^n$ 是列向量。于是

$$A = \begin{bmatrix} a_1^T \\ \vdots \\ a_n^T \end{bmatrix}, \quad Av = \begin{bmatrix} a_1^T v \\ \vdots \\ a_n^T v \end{bmatrix}, \quad A^T = \begin{bmatrix} a_1 & \cdots & a_n \end{bmatrix}.$$

因为 $a_i^T v$ 为实数, $a_i^T v = v^T a_i$, 所以

$$(Av)^T = \begin{bmatrix} a_1^T v & \cdots & a_n^T v \end{bmatrix} = \begin{bmatrix} v^T a_1 & \cdots & v^T a_n \end{bmatrix} = \boxed{v^T A^T}.$$

(b) 链式法则

这里求的是复合函数 $f(x(w), y(w))$ 对 w 的导数。首先

$$f_x = 2x + 2y, \quad f_y = 2x - 2y, \quad x' = 2w - 3, \quad y' = 4w^3 - 6w + 1.$$

由链式法则,

$$\begin{aligned} \frac{df}{dw} &= (2x + 2y)(2w - 3) + (2x - 2y)(4w^3 - 6w + 1) \\ &= 2(w^4 - 2w^2 - 2w)(2w - 3) + 2(-w^4 + 4w^2 - 4w)(4w^3 - 6w + 1) \\ &= \boxed{-8w^7 + 48w^5 - 40w^4 - 56w^3 + 60w^2 + 4w}. \end{aligned}$$

(c) 正交变换保持平方范数

正交矩阵满足 $U^T U = I$, 因此

$$\|Ux\|_2^2 = (Ux)^T (Ux) = x^T U^T U x = x^T x = \boxed{\|x\|_2^2}.$$

(d) 二次型的梯度

展开为 $f(x) = \sum_{i=1}^n \sum_{j=1}^n A_{ij} x_i x_j$ 。对 x_k 求偏导:

$$\frac{\partial f}{\partial x_k} = \sum_{j=1}^n A_{kj} x_j + \sum_{i=1}^n A_{ik} x_i.$$

第一项来自 x_i 的导数, 第二项来自 x_j 的导数; 当 $i = j = k$ 时, 两项合起来正好是 $2A_{kk}x_k$ 。故 $\nabla_x f = (A + A^T)x$ 。利用 $A = A^T$, 得到

$$\boxed{\nabla_x f(x) = 2Ax}.$$

Problem 2: 概率复习

(a) 恰好匹配 k 个号码

固定购买的彩票，令 K 表示它与中奖号码的匹配数。中奖号码是从 51 个号码中等概率选取的 6 元子集，共有 $\binom{51}{6}$ 种。

若恰好匹配 k 个，需要从彩票的 6 个号码中选出 k 个，再从其余 45 个号码中选出 $6 - k$ 个。两个选择共同且唯一地确定中奖集合，所以

$$\Pr(K = k) = \frac{\binom{6}{k} \binom{45}{6-k}}{\binom{51}{6}}, \quad k = 0, 1, \dots, 6.$$

这是超几何分布。注意号码不计顺序，因此使用组合数而非排列数。

(b) 机器说谎的概率

按通常约定，骰子为相互独立的公平六面骰，掷骰结果也独立于抽奖。令 L 表示机器说谎。

当 $K = 0$ 时，机器必定说实话，即 $\Pr(L | K = 0) = 0$ 。当 $K = k \geq 1$ 时， k 个骰子的点数之和至少为 k 。和等于 k 当且仅当每个骰子都是 1，因此

$$\Pr(L | K = k) = \left(\frac{1}{6}\right)^k, \quad k = 1, \dots, 6.$$

应用全概率公式，

$$\begin{aligned} \Pr(L) &= \sum_{k=0}^6 \Pr(L | K = k) \Pr(K = k) \\ &= \frac{1}{\binom{51}{6}} \sum_{k=1}^6 \binom{6}{k} \binom{45}{6-k} \frac{1}{6^k} \\ &= \frac{11992137545}{168049873152} \approx 0.071360586712. \end{aligned}$$

所以机器说谎的概率约为 7.1361%。求和从 $k = 1$ 开始，不能把 $k = 0$ 的情况也计作说谎。

Problem 3: 线性最小二乘

(a) 在特征向量坐标系中表示目标函数

由实对称矩阵的谱分解, $A = Q\Lambda Q^T$, 其中 $Q^T Q = Q Q^T = I$ 。定义

$$\tilde{x} = Q^T x, \quad \tilde{y} = Q^T y.$$

因为 $y = Q\tilde{y}$, 故

$$Ax - y = Q\Lambda Q^T x - Q Q^T y = Q(\Lambda\tilde{x} - \tilde{y}).$$

根据 Problem 1(c) 中的正交不变性,

$$\|Ax - y\|_2^2 = \|\Lambda\tilde{x} - \tilde{y}\|_2^2.$$

且 $x \leftrightarrow \tilde{x}$ 是一一对应的坐标变换, 逆变换为 $x = Q\tilde{x}$, 因此在两组坐标下最小化是等价的。

(b) 伪逆给出一个最小化解

令 $I_+ = \{i : \lambda_i \neq 0\}$, $I_0 = \{i : \lambda_i = 0\}$ 。这里 I_+ 仅表示非零指标, 并不要求特征值为正。逐坐标展开:

$$\|\Lambda\tilde{x} - \tilde{y}\|_2^2 = \sum_{i \in I_+} (\lambda_i \tilde{x}_i - \tilde{y}_i)^2 + \sum_{i \in I_0} \tilde{y}_i^2.$$

第二项与 $\tilde{x}$ 无关; 第一项非负, 且取 $\tilde{x}_i = \tilde{y}_i / \lambda_i$ 时每个平方项都为零。对 $i \in I_0$, $\tilde{x}_i$ 可以任意选择。特别地, 取它们为零, 则

$$\tilde{x}^* = \Lambda^\dagger \tilde{y}, \quad (\Lambda^\dagger)_{ii} = \begin{cases} 1/\lambda_i, & \lambda_i \neq 0, \\ 0, & \lambda_i = 0, \end{cases}$$

非对角元素全为零。变回原坐标得到

$$x^* = Q\Lambda^\dagger Q^T y, \quad \min_x f(x) = \sum_{i \in I_0} \tilde{y}_i^2.$$

解的非唯一性: 由于 A 不可逆, 至少一个零特征值存在。所有最小化解为

$$x = x^* + \sum_{i \in I_0} c_i q_i, \quad c_i \in \mathbb{R}.$$

这些额外项位于 $\ker A$, 不改变 Ax 。又由于它们与 x^* 正交, $\|x\|_2^2 = \|x^*\|_2^2 + \sum_{i \in I_0} c_i^2$, 所以 x^* 是唯一的最小范数最小二乘解。

(c) 加分题：一般矩阵的 SVD 解法

设 $A \in \mathbb{R}^{m \times n}$, $y \in \mathbb{R}^m$, $\text{rank}(A) = r$ 。取完整奇异值分解

$$A = U\Sigma V^T, \quad U \in \mathbb{R}^{m \times m}, \quad V \in \mathbb{R}^{n \times n}, \quad \Sigma \in \mathbb{R}^{m \times n},$$

其中 U, V 正交, Σ 的前 r 个对角元素为 $\sigma_i > 0$, 其余元素为零。定义 $z = V^T x \in \mathbb{R}^n$, $b = U^T y \in \mathbb{R}^m$ 。于是

$$\begin{aligned} \|Ax - y\|_2^2 &= \|U(\Sigma z - b)\|_2^2 = \|\Sigma z - b\|_2^2 \\ &= \sum_{i=1}^r (\sigma_i z_i - b_i)^2 + \sum_{i=r+1}^m b_i^2. \end{aligned}$$

这同时适用于高矩阵、宽矩阵和秩亏矩阵。后一个求和为无法通过选择 x 消去的常数；令

$$z_i^* = \begin{cases} b_i/\sigma_i, & 1 \leq i \leq r, \\ 0, & r < i \leq n, \end{cases}$$

便使第一个求和达到零。定义 $\Sigma^\dagger \in \mathbb{R}^{n \times m}$, 它的前 r 个对角元素为 $1/\sigma_i$, 其余为零。则

$$\boxed{x^* = V\Sigma^\dagger U^T y = A^\dagger y}, \quad \boxed{\min_x \|Ax - y\|_2^2 = \sum_{i=r+1}^m b_i^2}.$$

若 v_i 是 V 的第 i 列, 所有最小化解为

$$x = x^* + \sum_{i=r+1}^n c_i v_i, \quad c_i \in \mathbb{R}.$$

由 V 的正交性, x^* 仍是唯一的最小范数最小二乘解。当 $r = n$ 时最小化解唯一；当 $r < n$ 时不唯一。当 $r = 0$ 时, $A = 0$, 所有 x 都是最小化解, 而 $x^* = 0$ 是最小范数解。空求和按零处理, 因此上述公式也覆盖此情形。

Problem 4: NumPy 向量化

以下答案均不使用循环或列表推导式。附录是完成后的 notebook 的代码单元与实际运行输出；原 notebook 的参考检查函数保持不变。

(a) 各行的 ℓ_1 范数

```
def row_l1_norms(A):
    return np.abs(A).sum(axis=1)
```

对 $A \in \mathbb{R}^{N \times M}$ ，第 i 个输出为 $x_i = \sum_{j=1}^M |A_{ij}|$ 。绝对值逐元素计算，随后沿列方向（`axis=1`）求和，输出形状为 $(N,)$ 。函数体只有一行。

(b) 为每个住址分配最近基站

```
def closest_tower(A, B):
    a2 = (A**2).sum(axis=1)[None, :]
    b2 = (B**2).sum(axis=1)[:, None]
    sq_dists = b2 + a2 - 2 * B @ A.T
    return np.argmin(sq_dists, axis=1)
```

函数体恰好四行。假设有至少一个基站（ $M \geq 1$ ）。各个中间量的形状如下：

变量	形状	含义
<code>a2</code>	$(1, M)$	各基站坐标的平方范数
<code>b2</code>	$(N, 1)$	各住址坐标的平方范数
<code>B @ A.T</code>	(N, M)	所有住址与基站的内积
<code>sq_dists</code>	(N, M)	所有住址与基站的平方距离
输出	$(N,)$	各住址的最近基站索引

广播相加并减去内积项后，第 (i, j) 个元素为

$$\text{sq_dists}_{ij} = \|B_{i,:}\|_2^2 + \|A_{j,:}\|_2^2 - 2B_{i,:}A_{j,:}^T = \|B_{i,:} - A_{j,:}\|_2^2.$$

平方根严格递增，因此无需开平方就能找最近基站。沿 `axis=1` 在每行的 M 个基站中取最小值索引。距离相同时，`np.argmin` 返回第一个最小值的索引。

原代码有两处错误：内积前应为负号；`argmin` 应沿 `axis=1`，才能为每个住址返回一个索引。

附录：完成后的 notebook

以下按原顺序列出全部代码单元及运行输出。原题的检查单元包含参考循环，仅用于验证，不属于答案函数。

In [1]

```
import numpy as np

rng = np.random.default_rng(453)
```

In [2]

```
def row_l1_norms(A):
    return np.abs(A).sum(axis=1)
```

In [3]

```
# --- check (a) ---
A_test = rng.normal(size=(5, 4))

def _ref_row_l1(A):
    # slow reference, for checking only -- do not use in your
    answer
    return np.array([np.abs(A[i]).sum() for i in range(A.shape[0])])

out = row_l1_norms(A_test)
assert out.shape == (5,), f"expected shape (5,), got {out.shape}"
assert np.allclose(out, _ref_row_l1(A_test)), "values do not match"
print("(a) looks correct")
```

实际输出

```
(a) looks correct
```

In [4]

```
def closest_tower(A, B):
    a2 = (A**2).sum(axis=1)[None, :]
    b2 = (B**2).sum(axis=1)[:, None]
    sq_dists = b2 + a2 - 2 * B @ A.T
    return np.argmin(sq_dists, axis=1)
```

In [5]

```
# --- check (b) ---
A_towers = rng.normal(size=(7, 3))      # M = 7 towers, D = 3
B_homes = rng.normal(size=(11, 3))      # N = 11 homes

def _ref_closest(A, B):                  # slow reference, for checking only -- do not use in your
    answer
    return np.array([int(np.argmin((B[i] - A) ** 2).sum(axis=1))) for i in range(B.shape[0])
])

out = closest_tower(A_towers, B_homes)
assert out.shape == (B_homes.shape[0],), f"expected shape {(B_homes.shape[0],)}, got {out.
    shape}"
assert np.array_equal(out, _ref_closest(A_towers, B_homes)), "indices are not the nearest
    towers"
print("(b) looks correct")
```

实际输出

```
(b) looks correct
```